

Отчёт по лабораторной работе №5

Математические основы защиты информации и информационной безопасности

Ли Хан

Содержание

1	Цель работы	5
2	Подробное описание реализации алгоритмов	6
2.1	Тест Ферма	6
2.1.1	Тест Ферма компилятора	7
2.1.2	результат	8
2.2	Алгоритм вычисления символа Якоби	8
2.2.1	Алгоритм вычисления символа Якоби компилятора	9
2.3	Тест Соловья-Штрассена	10
2.3.1	Тест Соловья-Штрассена компилятора	11
2.3.2	результат	11
2.4	Тест Миллера-Рабина	12
2.4.1	Тест Миллера-Рабинаа компилятора	13
2.4.2	Тест Миллера-Рабинаа компилятора	14
2.4.3	результат	15
3	Вывод	16

Список иллюстраций

2.1	Тест Ферма компилятора	7
2.2	Тест Ферма результат	8
2.3	Алгоритм вычисления символа Якоби компилятора	9
2.4	Тест Соловья-Штрассена компилятора	11
2.5	Тест Соловья-Штрассена результат	11
2.6	Тест Миллера-Рабина компилятора	13
2.7	Тест Миллера-Рабина компилятора	14
2.8	Тест Миллера-Рабина результат	15

Список таблиц

1 Цель работы

Изучить и освоить вероятностные алгоритмы проверки чисел на простоту, широко используемые в криптографии. Реализовать программно тесты Ферма, Соловья-Штрассена и Миллера-Рабина, а также понять их основные принципы и вероятность ошибки.

2 Подробное описание реализации алгоритмов

2.1 Тест Ферма

Основан на малой теореме Ферма: для простого p выполняется $a^{p-1} \equiv 1 \pmod{p}$.

Шаги реализации:

- Вход: Нечетное целое число $n \geq 5$.
- Выбор основания: Выбрать случайное целое число a в диапазоне $2 \leq a \leq n - 2$.
- Вычисление: Вычислить $r = a^{n-1} \pmod{n}$.
- Проверка результата:
 - Если $r = 1$, результат: «Число n , вероятно, простое».
 - В противном случае результат: «Число n составное».

2.1.1 Тест Ферма компилятора

```
import random

def fermat_test(n, t=5):
    """
    实现费马素性检验。
    Реализация теста Ферма.
    """
    # 按照实验要求，n 必须是不小于 5 的奇数
    # Согласно требованиям, n должен
    if n < 5 or n % 2 == 0:
        return "Число n составное"

    for _ in range(t):
        # 1. 选择随机整数 a, 2 <= a <= n-2
        # 1. Выбрать случайное целое
        a = random.randint(2, n - 2)

        # 2. 计算 r = a^(n-1) mod n
        # 2. Вычислить r = a^(n-1) (mod n)
        r = pow(a, n - 1, n)

        # 3. 如果 r != 1, 则结果为“合数”
        # 3. При r != 1 результат: «Число
        if r != 1:
            return "Число n составное"

    # 如果所有测试通过，结果为“大概率是素数”
    # В противном случае: «Число n, в
    return "Число n, вероятно, простое"
```

Рис. 2.1: Тест Ферма компилятора

2.1.2 результат

```
# 样例 (Пример):  
# 19 是素数, 15 是合数 (19 - простое, 15 - составное)  
print(f"Fermat Test (n=19): {fermat_test(19)}")  
print(f"Fermat Test (n=15): {fermat_test(15)}")
```

```
Fermat Test (n=19): Число n, вероятно, простое  
Fermat Test (n=15): Число n составное
```

Рис. 2.2: Тест Ферма результат

2.2 Алгоритм вычисления символа Якоби

Инструмент для определения квадратичных вычетов, вычисляемый через рекурсивное применение закона взаимности.

Шаги реализации:

- Инициализация: Положить $g = 1$.
- Обработка значений: При $a = 0$ результат 0; при $a = 1$ результат g .
- Выделение множителя 2: Представить a в виде $a = 2^k \cdot a_1$, где число a_1 нечетное.
- Обработка множителей: При нечетном k : $s = 1$, если $n \equiv \pm 1 \pmod{8}$; $s = -1$, если $n \equiv \pm 3 \pmod{8}$. При четном k положить $s = 1$.
- Закон взаимности: Если $n \equiv 3 \pmod{4}$ и $a_1 \equiv 3 \pmod{4}$, то $s = -s$.
- Итерация: Положить $a = n \pmod{a_1}$, $g = g \cdot s$ и вернуться к шагу 2.

2.2.1 Алгоритм вычисления символа Якоби компилятора

```
import random

def jacobi_symbol(a, n):
    """
    计算雅可比符号 (a/n) 的算法。
    Алгоритм вычисления символа Якоби (a/n).
    """
    # 1. 设置 g = 1
    # 1. Положить g = 1
    g = 1

    while a != 0:
        # 2, 3. 处理 a=0 和 a=1 的情况
        # 2, 3. При a=0 результат: 0; при a=1 результат: g
        if a == 0: return 0
        if a == 1: return g

        # 4. 将 a 表示为 2^k * a1, 其中 a1 是奇数
        # 4. Представить a в виде a = 2^k * a1, где a1 нечетное
        k = 0
        a1 = a
        while a1 % 2 == 0:
            k += 1
            a1 //= 2

        # 5. 根据 k 的奇偶性和 n mod 8 更新符号 s
        # 5. При нечетном k обновить s в зависимости от n mod 8
        if k % 2 == 1:
            if n % 8 in [1, 7]:
                s = 1
            else:
                s = -1
            g *= s

        # 6. 如果 a1 = 1, 返回结果 g * s
        # 6. При a1 = 1 результат: g * s
        if a1 == 1:
            return g

        # 7. 如果 n = 3 mod 4 且 a1 = 3 mod 4, 则反转符号
        # 7. Если n = 3 (mod 4) и a1 = 3 (mod 4), то s = -s
        if n % 4 == 3 and a1 % 4 == 3:
            g = -g

        # 8. 更新 a 和 n, 循环计算: a = n mod a1, n = a1
        # 8. Положить a = n mod a1, g = g * s и вернуться на шаг 3
        a, n = n % a1, a1

    return g
```

Рис. 2.3: Алгоритм вычисления символа Якоби компилятора

2.3 Тест Соловья-Штрассена

Использует критерий Эйлера, сравнивая результат возведения в степень с символом Якоби.

Шаги реализации:

- Вход: нечетное $n \geq 5$.
- Выбрать случайное $a \in [2, n - 2]$.
- Вычислить $r = a^{(n-1)/2} \pmod{n}$.
- Если $r \neq 1$ и $r \neq n - 1$, число составное.
- Вычислить символ Якоби $s = \left(\frac{a}{n}\right)$.
- Если $r \equiv s \pmod{n}$, число вероятно простое; иначе — составное.

2.3.1 Тест Соловья-Штрассена компилятора

```
def solovay_strassen(n, t=5):
    """
    实现索洛维-斯特拉森素性检验。
    Реализация теста Соловья-Штрассена.
    """
    if n < 5 or n % 2 == 0:
        return "Число n составное"

    for _ in range(t):
        # 1. 选择随机整数 a, 2 <= a < n-2
        # 1. Выбрать случайное целое число a, 2 <= a < n-2
        a = random.randint(2, n - 2)

        # 2. 计算 r = a^((n-1)/2) mod n
        # 2. Вычислить r = a^((n-1)/2) (mod n)
        r = pow(a, (n - 1) // 2, n)

        # 3. 如果 r != 1 且 r != n-1, 结果为“合数”
        # 3. При r != 1 и r != n-1 результат: «Число n составное»
        if r != 1 and r != n - 1:
            return "Число n составное"

        # 4. 计算雅可比符号 s = (a/n)
        # 4. Вычислить символ Якоби s = (a/n)
        s = jacobi_symbol(a, n) % n

        # 5. 如果 r 不等于 s mod n, 则为合数
        # 5. При r != s (mod n) результат: «Число n составное»
        if r != s:
            return "Число n составное"

    # 结果为大概率素数
    # Результат: «Число n, вероятно, простое»
    return "Число n, вероятно, простое"
```

Рис. 2.4: Тест Соловья-Штрассена компилятора

2.3.2 результат

```
# 样例 (Пример):
# 17 是素数, 25 是合数 (17 - простое, 25 - составное)
print(f"Solovay-Strassen (n=17): {solovay_strassen(17)}")
print(f"Solovay-Strassen (n=25): {solovay_strassen(25)}")
```

```
Solovay-Strassen (n=17): Число n, вероятно, простое
Solovay-Strassen (n=25): Число n составное
```

Рис. 2.5: Тест Соловья-Штрассена результат

2.4 Тест Миллера-Рабина

Самый эффективный современный тест, основанный на поиске нетривиальных корней из единицы.

Шаги реализации:

- Разложить $n - 1 = 2^s \cdot r$, где r нечетное.
- Выбрать случайное $a \in [2, n - 2]$.
- Вычислить $y = a^r \pmod n$.
- Если $y \neq 1$ и $y \neq n - 1$: возводить y в квадрат до $s - 1$ раз.
- Если y становится 1 в процессе — составное; если в конце $y \neq n - 1$ — составное.
- Иначе — число вероятно простое.

2.4.1 Тест Миллера-Рабина компилятора

```
import random

def miller_rabin(n, t=5):
    """
    实现米勒-拉宾素性检验。
    Реализация теста Миллера-Рабина.
    """
    if n < 5 or n % 2 == 0:
        return "Число n составное"

    # 1. 寻找  $n-1 = 2^s * r$ , 其中  $r$  是奇数
    # 1. Представить  $n-1$  в виде  $n-1 = 2^s * r$  :
    s = 0
    r = n - 1
    while r % 2 == 0:
        s += 1
        r //= 2
```

Рис. 2.6: Тест Миллера-Рабина компилятора

2.4.2 Тест Миллера-Рабина компилятора

```
for _ in range(t):
    # 2. 选择随机数 a,  $2 \leq a < n-2$ 
    # 2. ВЫБРАТЬ СЛУЧАЙНОЕ ЦЕЛОЕ ЧИСЛО a,  $2 \leq$ 
    a = random.randint(2, n - 2)

    # 3. 计算  $y = a^r \bmod n$ 
    # 3. ВЫЧИСЛИТЬ  $y = a^r \pmod n$ 
    y = pow(a, r, n)

    # 4. 如果 y 不等于 1 且不等于 n-1, 进行平方计算
    # 4. ПРИ  $y \neq 1$  И  $y \neq n-1$  ВЫПОЛНИТЬ СЛЕДУЮЩИ
    if y != 1 and y != n - 1:
        j = 1
        # 4.2. 循环直到  $j = s-1$  或  $y = n-1$ 
        # 4.2. ЕСЛИ  $j \leq s-1$  И  $y \neq n-1$ , ТО:
        while j <= s - 1 and y != n - 1:
            # 4.2.1. 计算  $y = y^2 \bmod n$ 
            # 4.2.1. ПОЛОЖИТЬ  $y = y^2 \pmod n$ 
            y = pow(y, 2, n)

            # 4.2.2. 如果  $y = 1$ , 说明是合数
            # 4.2.2. ПРИ  $y = 1$  РЕЗУЛЬТАТ: «Число
            if y == 1:
                return "Число n составное"

            j += 1

        # 4.3. 如果最终 y 仍不等于 n-1, 则结果为“合数”
        # 4.3. ПРИ  $y \neq n-1$  РЕЗУЛЬТАТ: «Число n со
        if y != n - 1:
            return "Число n составное"
```

Рис. 2.7: Тест Миллера-Рабина компилятора

2.4.3 результат

```
# 样例 (Пример):  
# 23 是素数, 21 是合数 (23 - простое, 21 - составное)  
print(f"Miller-Rabin (n=23): {miller_rabin(23)}")  
print(f"Miller-Rabin (n=21): {miller_rabin(21)}")
```

```
Miller-Rabin (n=23): Число n, вероятно, простое  
Miller-Rabin (n=21): Число n составное
```

Рис. 2.8: Тест Миллера-Рабина результат

3 Вывод

Реализованы три алгоритма. Установлено, что точность повышается с количеством тестов t , а вероятность ошибки не превышает $1/2^t$. Тест Миллера-Рабина является наиболее надежным.